

LAPORAN PRAKTIKUM

JOBSHEET 5 - JAVASCRIPT DOM DAN EVENT



Disusun Oleh:

Nama : Mohammad Hafidz Rafi' Rabbani
NIM : 254107020084
Kelas : 2H
No. Absen : 14
Repository : github.com/hafidzrafi/DnPWeb2026

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2026

Daftar Isi

1. Tujuan	3
2. Praktikum	3
2.1. Percobaan 1: Menu Hamburger dengan JavaScript	3
2.1.1. 1.1 Analisis Mekanisme classList.toggle	3
2.1.2. 1.2 Keunggulan Arsitektur Dibanding Checkbox Hack	4
2.1.3. Langkah-langkah	5
2.1.4. Pertanyaan	7
2.2. Percobaan 2: Konfirmasi Hapus Data Front-End	8
2.2.1. 2.1 Mekanisme DOM Traversal closest() dan remove()	8
2.2.2. Langkah-langkah	9
2.2.3. Pertanyaan	10
2.3. Percobaan 3: Filter dan Pencarian Tabel Real-Time	11
2.3.1. 3.1 Implementasi Event Listener keyup	11
2.3.2. 3.2 Analisis String Matching & Display Toggle	11
2.3.3. Langkah-langkah	12
2.3.4. Pertanyaan	13
2.4. Percobaan 4: Validasi Form Sisi Klien (Client-Side Validation)	14
2.4.1. 4.1 Konstruksi Pesan Error Dinamis	14
2.4.2. 4.2 Pola Clean-State & Pembatalan Submit (e.preventDefault)	14
2.4.3. Langkah-langkah	16
2.4.4. Pertanyaan	17
3. Tugas	18
3.1. Tugas Evaluasi & Analisis Konseptual Jobsheet 5	18

1. Tujuan

- Memahami konsep dasar Document Object Model (DOM) sebagai pohon objek (tree hierarchy) di memori browser yang merepresentasikan dokumen HTML.
- Menguasai teknik seleksi elemen DOM menggunakan metode modern seperti `document.getElementById()` dan `document.querySelectorAll()`.
- Menerapkan pemrograman berbasis kejadian (event-driven programming) melalui penambahan Event Listener pada elemen antarmuka.
- Menggantikan teknik CSS Checkbox Hack dengan manipulasi status antarmuka berbasis `classList.toggle()` pada menu navigasi responsif (hamburger menu).
- Mengimplementasikan konfirmasi interaktif penghapusan baris data tabel secara lokal di sisi klien menggunakan DOM traversal `closest()` dan method `remove()`.
- Membangun fitur pencarian dan penyaringan data tabel secara real-time berdasarkan input keyboard pengguna (event `keyup/input`) dan manipulasi properti `style.display`.
- Merancang sistem validasi form di sisi klien (client-side validation) dengan pencegahan aksi bawaan form via `e.preventDefault()` serta penyisipan pesan error dinamis menggunakan `insertAdjacentElement()`.
- Menyimpan dan mendokumentasikan seluruh source code pekerjaan pada repositori GitHub resmi: <https://github.com/hafidzrafi/DnPWeb2026>.

2. Praktikum

2.1. Percobaan 1: Menu Hamburger dengan JavaScript

Pada percobaan pertama, mekanisme navigasi mobile (**hamburger menu**) yang sebelumnya diimplementasikan pada Jobsheet 3 menggunakan teknik **Checkbox Hack** CSS murni (`#nav-toggle:checked ~ nav`) kini dimigrasikan menggunakan tombol semantik `<button id="nav-toggle-btn">` yang dikendalikan oleh fungsi JavaScript `initNavToggle()`.

2.1.1. 1.1 Analisis Mekanisme `classList.toggle`

Fungsi `initNavToggle()` bekerja dengan mencari elemen tombol pemicu dan elemen `<nav>` di dalam `<header>`. Saat pengguna menekan tombol, method `nav.classList.toggle("nav-open")` memeriksa apakah class `.nav-open` sudah terpasang pada elemen navigasi. Jika belum ada maka ditambahkan, dan jika sudah ada maka dicabut:

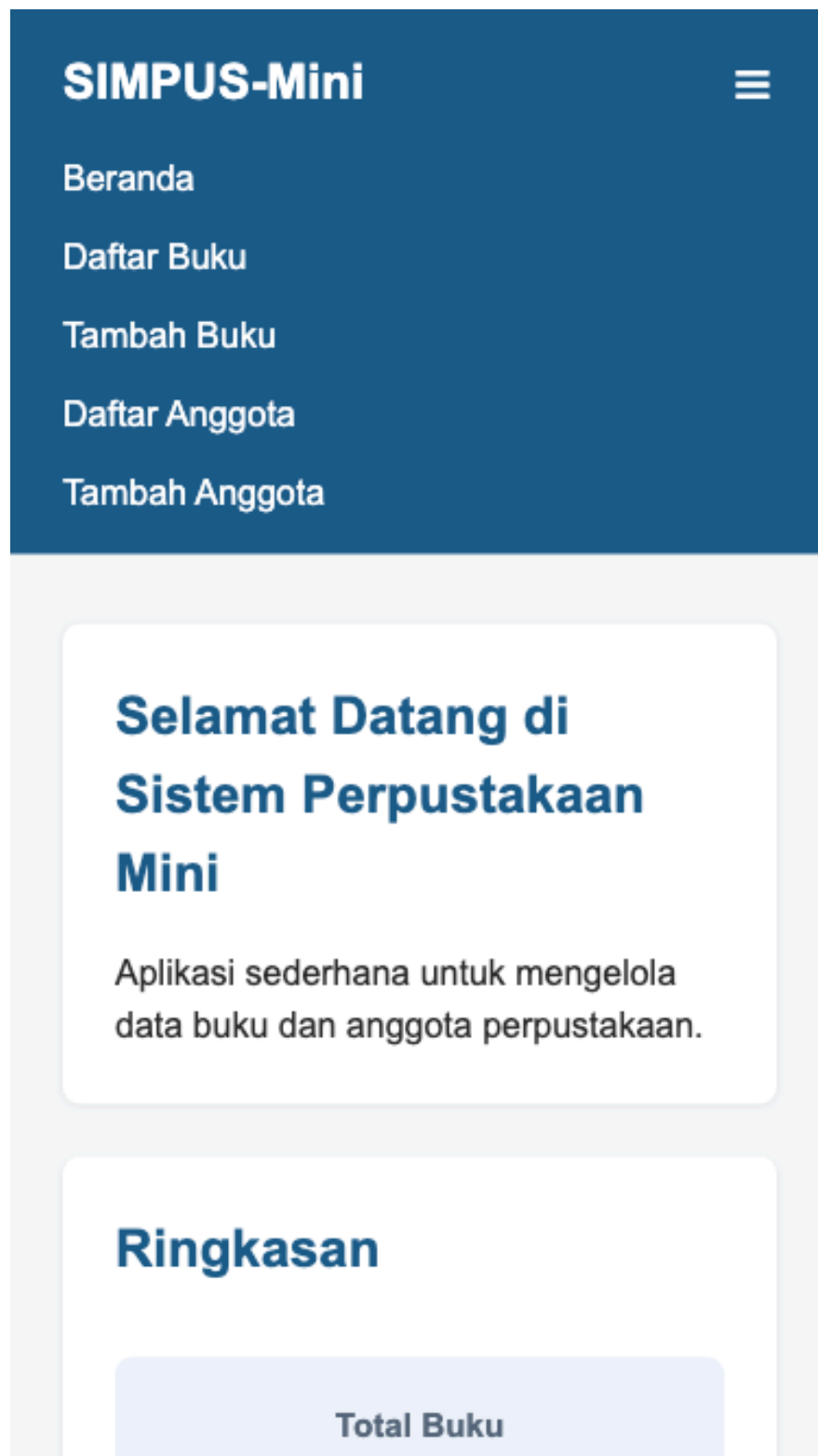
```
function initNavToggle() {
  const toggleBtn = document.getElementById("nav-toggle-btn");
  const nav = document.querySelector("header nav");
  if (!toggleBtn || !nav) return;

  toggleBtn.addEventListener("click", function() {
    nav.classList.toggle("nav-open");
  });
}
```

2.1.2. 1.2 Keunggulan Arsitektur Dibanding Checkbox Hack

Migrasi ke JavaScript memberikan dua keunggulan rekayasa yang mendasar:

1. **Aksesibilitas dan Semantik Web (A11y):** Menggunakan elemen `<button>` resmi yang secara bawaan dapat menerima fokus keyboard (tombol Tab), dapat diaktifkan menggunakan tombol Spasi/Enter, dan terbaca secara akurat oleh pembaca layar (**screen reader**).
2. **Pemisahan Peran (Separation of Concerns):** Menghilangkan ketergantungan rapuh pada selector CSS saudara (`~`) dan pseudo-class `:checked`. CSS kini murni bertugas mengatur tata letak visual elemen yang memiliki class `.nav-open` (`display: block`).



Gambar 1.1: Antarmuka Navigasi Mobile Terbuka saat Tombol Hamburger Diklik
(`nav.classList.toggle("nav-open")`).

2.1.3. Langkah-langkah

1. Membuat berkas JavaScript terpusat pada direktori `assets/js/app.js`.

2. Menggantikan elemen `<input type="checkbox" id="nav-toggle">` pada header HTML dengan `<button id="nav-toggle-btn" class="nav-toggle-label">≡</button>`.
3. Menghubungkan berkas script menggunakan tag `<script src="../assets/js/app.js"></script>` tepat sebelum penutup tag `</body>`.
4. Menguji klik tombol hamburger pada viewport mobile (lebar 375px) dan memverifikasi kemunculan menu vertikal.

2.1.4. Pertanyaan

1. Mengapa penempatan tag `<script>` direkomendasikan diletakkan tepat sebelum penutup `</body>` ?
 - Penempatan tag `<script>` di akhir `</body>` mencegah terjadinya **parser blocking**. Ketika browser mem-parse dokumen HTML dari atas ke bawah, penempatan script di dalam `<head>` akan menghentikan pembentukan DOM untuk mengunduh dan mengeksekusi JavaScript. Jika script mengakses elemen halaman saat DOM belum selesai dibangun, fungsi seperti `document.getElementById()` akan menghasilkan nilai `null` dan memicu error **TypeError**. Meletakkan script sebelum `</body>` memastikan seluruh elemen antarmuka selesai dirender terlebih dahulu sehingga halaman terasa cepat dimuat (**First Contentful Paint** optimal).

2.2. Percobaan 2: Konfirmasi Hapus Data Front-End

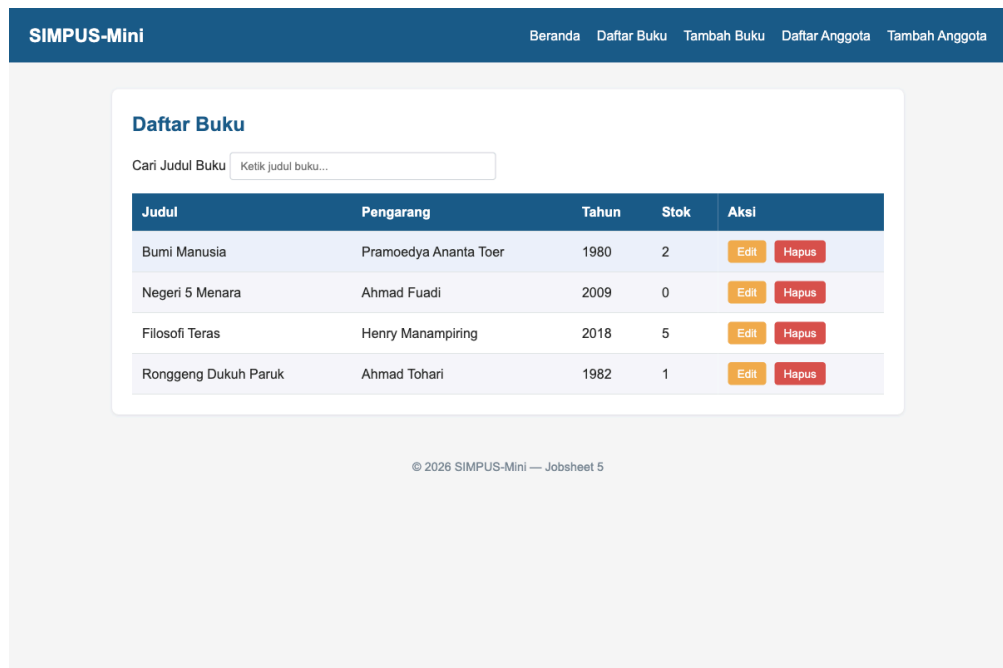
Percobaan kedua menerapkan interaktivitas konfirmasi penghapusan data pada tabel buku dan anggota. Karena backend database baru akan diperkenalkan pada Jobsheet 8, aksi penghapusan pada Jobsheet 5 ini beroperasi di level **front-end DOM manipulation**.

2.2.1. 2.1 Mekanisme DOM Traversal `closest()` dan `remove()`

Saat pengguna menekan tombol dengan class `.btn-hapus`, event listener menangkap target klik dan menelusuri hierarki pohon DOM ke atas (**ancestor traversal**) menggunakan method `btn.closest("tr")`. Method ini secara cerdas mencari elemen baris tabel pembungkus terdekat tanpa peduli berapa lapis tag yang membungkus tombol tersebut:

```
function initHapusConfirm() {
  document.querySelectorAll(".btn-hapus").forEach(function(btn) {
    btn.addEventListener("click", function() {
      const row = btn.closest("tr");
      const nama = row ? row.querySelector("td").textContent : "data ini";
      const yakin = confirm("Yakin ingin menghapus \"" + nama + "\"?");
      if (yakin && row) {
        row.remove();
      }
    });
  });
}
```

Method bawaan browser `confirm()` menampilkan dialog modal konfirmasi sinkron yang memblokir eksekusi sementara hingga pengguna memilih OK (`true`) atau Batal (`false`). Jika disetujui, `row.remove()` mencabut elemen `<tr>` secara langsung dari Document Object Model, memicu proses **reflow** dan **repaint** pada browser.



Gambar 1.2: Baris Pertama Tabel Berhasil Dihapus dari DOM setelah Konfirmasi Dialog `confirm()` Disetujui.

2.2.2. Langkah-langkah

1. Menambahkan class `.btn-hapus` pada setiap tombol aksi Hapus di tabel daftar buku dan anggota.
2. Mengimplementasikan fungsi `initHapusConfirm()` dengan perulangan `querySelectorAll().forEach()`.
3. Menguji penekanan tombol Hapus pada salah satu baris buku dan memverifikasi dialog konfirmasi serta hilangnya baris dari layar.

2.2.3. Pertanyaan

1. Apa perbedaan mendasar antara menghapus elemen menggunakan `row.remove()` dibandingkan menyembunyikannya menggunakan `row.style.display = "none"`?
 - `row.remove()` secara fisik mencabut node elemen beserta seluruh turunan dan event listener-nya dari memori Document Object Model (DOM tree). Elemen tersebut benar-benar hilang dari struktur dokumen. Sebaliknya, `row.style.display = "none"` hanya mengubah properti tata letak visual (**rendering tree**) sehingga elemen tidak digambar di layar, namun node tersebut tetap ada di dalam memori DOM dan tetap bisa ditemukan melalui query selector JavaScript.

2.3. Percobaan 3: Filter dan Pencarian Tabel Real-Time

Percobaan ketiga membangun fitur pencarian instan pada tabel tanpa memicu pemuatan ulang halaman (**zero page-reload**). Pengguna cukup mengetik kata kunci pada kolom input, dan baris tabel yang tidak sesuai akan disaring secara otomatis seiring tombol keyboard ditekan.

2.3.1. 3.1 Implementasi Event Listener keyup

Fungsi `initTableFilter()` mendengarkan kejadian keyboard pada `#search-input`:

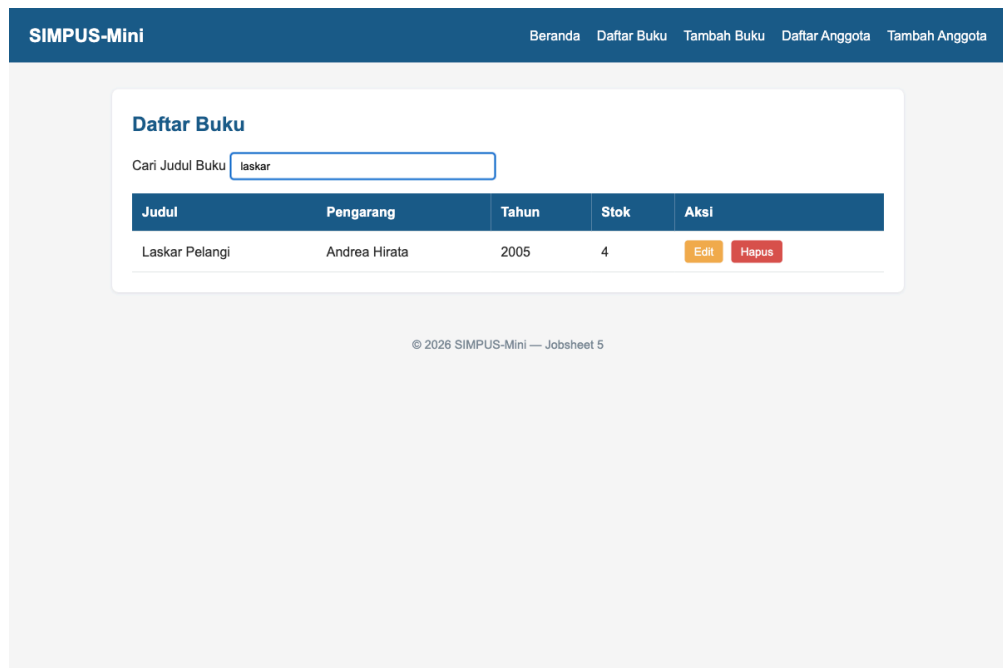
```
function initTableFilter() {
  const input = document.getElementById("search-input");
  const table = document.querySelector(".table-responsive table");
  if (!input || !table) return;

  input.addEventListener("keyup", function() {
    const keyword = input.value.toLowerCase();
    const rows = table.querySelectorAll("tbody tr");
    rows.forEach(function(row) {
      const teks = row.textContent.toLowerCase();
      row.style.display = teks.includes(keyword) ? "" : "none";
    });
  });
}
```

2.3.2. 3.2 Analisis String Matching & Display Toggle

Properti `row.textContent` secara otomatis menggabungkan seluruh teks dari kolom Judul, Pengarang, Tahun, hingga Kategori pada baris tersebut. Normalisasi string menggunakan `.toLowerCase()` memastikan pencarian bersifat **case-insensitive**.

Saat kata kunci cocok (`teks.includes(keyword)` bernilai `true`), baris diatur ke `display = ""` agar kembali ke aturan stylesheet default (`display: table-row`), bukan dipaksa menjadi `display: block` yang berpotensi merusak tata letak kisi tabel HTML.



Gambar 1.3: Hasil Penyaringan Real-Time Kata Kunci “laskar” Menampilkan Buku Terkait dan Menyembunyikan Baris Lainnya.

2.3.3. Langkah-langkah

1. Menambahkan elemen input pencarian `<input type="search" id="search-input">` di atas tabel data.
2. Mengimplementasikan fungsi `initTableFilter()` yang mendengarkan event keyboard.
3. Mengetikkan berbagai kata kunci (sebagian kata, huruf besar/kecil) dan mengamati perilaku penyaringan baris tabel secara real-time.

2.3.4. Pertanyaan

1. Mengapa pada kasus pencarian teks modern, event `input` lebih direkomendasikan daripada event `keyup`?
 - Event `keyup` hanya terpicu saat tombol fisik pada keyboard dilepas. Jika pengguna memasukkan teks dengan cara melakukan klik kanan lalu memilih **Paste** menggunakan mouse, memilih kata dari saran **autocomplete**, atau menekan tombol silang pembersih teks pada input pencarian, event `keyup` tidak akan pernah terpicu sehingga tabel gagal tersaring. Sebaliknya, event `input` dirancang untuk menangkap setiap perubahan nilai masukan (**value mutation**) tanpa peduli dari mana sumber aksi tersebut berasal.

2.4. Percobaan 4: Validasi Form Sisi Klien (Client-Side Validation)

Percobaan keempat mengamankan integritas input pengguna sebelum data dikirimkan. Form Tambah Buku dan Tambah Anggota diproteksi dengan aturan validasi field wajib, rentang tahun terbit (1900–2026), serta kuantitas stok non-negatif.

2.4.1. 4.1 Konstruksi Pesan Error Dinamis

Pesan error dibuat secara dinamis menggunakan `document.createElement("span")` dengan class CSS `.error` dan disisipkan tepat di bawah field yang bermasalah menggunakan method `insertAdjacentElement("afterend", span)`:

```
function tampilkanError(input, pesan) {
    hapusError(input);
    const span = document.createElement("span");
    span.className = "error";
    span.textContent = pesan;
    input.insertAdjacentElement("afterend", span);
}

function hapusError(input) {
    const next = input.nextElementSibling;
    if (next && next.classList.contains("error")) {
        next.remove();
    }
}
```

2.4.2. 4.2 Pola Clean-State & Pembatalan Submit (e.preventDefault)

Fungsi `tampilkanError()` selalu mengeksekusi `hapusError(input)` terlebih dahulu untuk mencegah duplikasi pesan error bertumpuk saat tombol submit ditekan berkali-kali.

Apabila terdapat minimal satu field yang tidak valid (`valid === false`), method `e.preventDefault()` wajib dipanggil untuk membatalkan pengiriman HTTP dan refresh halaman bawaan browser:

SIMPUS-Mini Beranda Daftar Buku Tambah Buku Daftar Anggota Tambah Anggota

Tambah Buku

Judul

Field ini wajib diisi.

Pengarang

Field ini wajib diisi.

Tahun Terbit

Tahun harus di antara 1900-2026.

ISBN

Stok

Stok tidak boleh negatif.

Kategori

Fiksi ▾

Simpan

Gambar 1.4: Tampilan Pesan Error Validasi Inline pada Form Tambah Buku saat Terjadi Pelanggaran Validasi Input.

SIMPUS-Mini Beranda Daftar Buku Tambah Buku Daftar Anggota Tambah Anggota

Tambah Anggota

Nama

Field ini wajib diisi.

No. Anggota

Alamat

No. HP

Simpan

© 2026 SIMPUS-Mini — Jobsheet 5

Gambar 1.5: Tampilan Pesan Error Validasi Inline pada Form Tambah Anggota saat Field Nama Dikosongkan.

2.4.3. Langkah-langkah

1. Menambahkan atribut `id="form-tambah"` pada tag `<form>` di `books/tambah.html` dan `members/tambah.html`.
2. Menambahkan aturan gaya CSS untuk class `.error` dengan warna teks merah semantik `#d9534f`.
3. Mengimplementasikan fungsi pembantu `tampilkanError()`, `hapusError()`, dan fungsi pengendali utama `initValidasiForm()`.
4. Menguji pengiriman form dengan field kosong, angka tahun di luar rentang, dan stok negatif untuk memverifikasi penolakan form serta kemunculan pesan error inline.

2.4.4. Pertanyaan

1. Apa yang terjadi apabila `e.preventDefault()` tidak dipanggil saat form mendeteksi data yang tidak valid?
 - Jika `e.preventDefault()` tidak dipanggil, browser akan tetap menjalankan aksi bawaan (**default submit action**) yaitu merefresh/memuat ulang halaman. Akibatnya, elemen pesan error yang baru saja disisipkan oleh JavaScript akan langsung musnah seketika dan seluruh isian form akan tereset ke kondisi semula, sehingga pengguna tidak dapat membaca alasan mengapa data yang mereka masukkan ditolak.

3. Tugas

3.1. Tugas Evaluasi & Analisis Konseptual Jobsheet 5

Berikut adalah evaluasi kritis dan pembahasan mendalam terhadap lima pertanyaan evaluasi konsep pada Jobsheet 5:

No	Pertanyaan dan Jawaban Konseptual
1	Soal: Mengapa penempatan tag <code><script src="..."></code> direkomendasikan diletakkan tepat sebelum tag penutup <code></body></code> dan bukan di dalam <code><head></code>? Jawaban: Proses parsing dokumen HTML oleh mesin peramban (browser engine) berlangsung secara sekuensial dari baris teratas ke baris terbawah. Ketika tag <code><script></code> diletakkan di dalam <code><head></code>, browser mengalami parser blocking di mana pembentukan pohon DOM dihentikan sementara hingga berkas JavaScript selesai diunduh dan dieksekusi. Karena elemen-elemen di dalam tag <code><body></code> belum terbaca, pemanggilan selektor seperti <code>document.getElementById("form-tambah")</code> akan mengembalikan nilai <code>null</code>. Menempatkan tag script tepat sebelum penutup <code></body></code> menjamin seluruh Document Object Model telah terkonstruksi secara utuh, mempercepat metrik First Contentful Paint (FCP), serta mencegah timbulnya runtime error. Alternatif modern pada web engineering adalah meletakkannya di <code><head></code> dengan menambahkan atribut <code>defer</code> agar pengunduhan berjalan asinkron dan dieksekusi tepat sebelum event <code>DOMContentLoaded</code>.
2	Soal: Bagaimana cara kerja fungsi <code>initNavToggle()</code> menggunakan <code>classList.toggle("nav-open")</code> dan apa kelebihanannya dibandingkan teknik Checkbox Hack CSS murni pada Jobsheet 3? Jawaban: Fungsi <code>initNavToggle()</code> memasang penanganan event <code>click</code> pada elemen tombol <code>#nav-toggle-btn</code>. Setiap kali tombol ditekan, method <code>classList.toggle("nav-open")</code> mengevaluasi keberadaan

No	Pertanyaan dan Jawaban Konseptual
	class <code>.nav-open</code> pada elemen <code><nav></code>. Jika class telah ada maka akan dicabut, dan jika belum ada maka akan ditambahkan, sehingga memicu aturan CSS <code>header nav.nav-open { display: block; }</code>. Kelebihan utama pendekatan ini meliputi: 1. Aksesibilitas Semantik (A11y): Menggunakan tag semantik <code><button></code> yang dapat diakses penuh melalui keyboard (navigasi Tab, tombol Enter/Spasi) dan dikenali oleh teknologi asistif (screen reader). 2. Pemisahan Perhatian (Separation of Concerns): Memisahkan state interaksi (dikelola oleh JavaScript) dari aturan presentasi visual (dikelola oleh CSS), serta menyederhanakan kode dengan mengeliminasi selector kombinator saudara (<code>~</code>) yang rumit pada Checkbox Hack.
3	Soal: Bagaimana mekanisme event <code>keyup</code> pada <code>initTableFilter()</code> menyaring baris data tabel secara real-time? Jelaskan penggunaan properti <code>textContent</code> dan manipulasi CSS <code>style.display</code>. Jawaban: Setiap kali pengguna melepaskan tombol keyboard di dalam <code>#search-input</code>, event <code>keyup</code> terpicu. Fungsi mengambil nilai input dan menormalisasinya menjadi huruf kecil melalui method <code>.toLowerCase()</code>. Selanjutnya, fungsi melakukan perulangan pada seluruh baris <code><tr></code> di dalam <code><tbody></code>. Properti <code>row.textContent</code> mengekstrak seluruh gabungan teks dari semua sel <code><td></code> pada baris tersebut. String teks ini juga dinormalisasi ke huruf kecil lalu diuji kecocokannya menggunakan method <code>.includes(keyword)</code>. Apabila cocok, tampilan baris dikembalikan dengan menyetel <code>row.style.display = ""</code> (mengikuti aturan stylesheet default yaitu <code>display: table-row</code>). Apabila tidak cocok, baris disembunyikan menggunakan <code>row.style.display = "none"</code>.
4	Soal: Mengapa method <code>e.preventDefault()</code> wajib dipanggil saat proses validasi form (<code>initValidasiForm</code>) mendeteksi adanya data

No	Pertanyaan dan Jawaban Konseptual
	input yang tidak valid? Apa yang terjadi jika method tersebut tidak dipanggil? Jawaban: Perilaku standar (default behavior) dari event <code>submit</code> pada elemen <code><form></code> adalah mengirimkan data ke URL target dan memicu pemuatan ulang (reload) halaman. Jika <code>e.preventDefault()</code> tidak dipanggil ketika input terbukti tidak valid, browser akan tetap mengeksekusi reload halaman. Akibatnya, seluruh elemen pesan kesalahan <code></code> yang baru saja disisipkan ke dalam pohon DOM akan langsung musnah seketika dan form kembali kosong. Pemanggilan <code>e.preventDefault()</code> membatalkan aksi reload tersebut sehingga pengguna tetap berada pada halaman aktif dan dapat membaca serta memperbaiki kesalahan inputnya.
5	Soal: Mengapa validasi sisi klien (client-side validation) menggunakan JavaScript hanya dianggap sebagai lapisan kenyamanan pengguna (user experience) dan bukan sebagai lapisan keamanan data utama? Jawaban: Validasi sisi klien dieksekusi seluruhnya di dalam lingkungan peramban pengguna (untrusted execution environment). Kode JavaScript berada di bawah kontrol penuh pengguna dan dapat dengan sangat mudah dilewati melalui berbagai teknik: mematikan fitur JavaScript di browser, memanipulasi kode lewat DevTools Console, menghapus atribut validasi via Elements Inspector, atau mengirimkan request payload secara langsung menggunakan perangkat lunak cURL, Postman, atau skrip otomatisasi. Oleh karena itu, validasi client-side hanya berfungsi meningkatkan kenyamanan antarmuka (immediate feedback UX). Keamanan data yang sejati wajib dijamin oleh Validasi Sisi Server (Server-Side Validation) yang tidak dapat dimanipulasi oleh klien, yang akan diimplementasikan secara komprehensif mulai Jobsheet 7.